Project Title: Securely

Supervised By
Afsana Begum
Assistant Professor & Coordinator M.Sc
Department of Software Engineering
Daffodil International University

Submitted By
S. M Al Fuad Nur
ID: 0242220005341056
Department of Software Engineering
Daffodil International University

This Project report has been submitted in fulfillment of the requirements for the Degree of Bachelor of Science in Software Engineering.

Acknowledgement

First and foremost, I would like to express my sincere gratitude to Almighty Allah for giving me the strength, patience, and opportunity to successfully complete my project titled, "**Securely**".

I would like to convey my heartfelt appreciation and deepest gratitude to my respected supervisor, **Afsana Begum**, Assistant Professor & Coordinator M.Sc, Department of Software Engineering at Daffodil International University. The completion of this project has been made possible by her continuous guidance, unending patience, valuable suggestions, and energetic supervision. I am truly grateful for her insightful advice and constructive feedback throughout the entire development process.

I would also like to extend my sincere thanks to the respected teachers and staff of the SWE Department at Daffodil International University, as well as to Dr. Imran Mahmud, Head and Associate Professor, for their gracious assistance and for providing an excellent academic environment.

Working on this project has profoundly enhanced my practical knowledge and technical skills in software engineering, application architecture, and development. I want to express my gratitude to all of my course colleagues and friends who engaged in insightful discussions and offered their collaborative support while finishing the assigned material.

Lastly, I must respectfully and deeply thank my parents and family for their unending encouragement, prayers, and patience throughout my entire academic journey.

Abstract

Securely is an all-in-one cross-platform Flutter package that consolidates comprehensive security functionality into a single, lightweight solution for modern mobile, desktop, and web applications. It addresses the fragmented nature of the current Flutter security ecosystem by providing a unified API for critical protection layers.

The package delivers complete Runtime Application Self-Protection (RASP) capabilities, including real-time detection of debuggers, root/jailbreak status, emulators/simulators, Frida instrumentation, VPN usage, developer mode, USB debugging, and screen recording/casting. These telemetry checks invoke low-level OS-specific commands—such as *sysctl* calls in iOS or root directory scans in Android—to identify environmental threats.

Furthermore, **Securely** provides hardware-backed secure storage through **StoreSecurely**, supporting configurable AES-GCM and AES-CBC encryption using native platform keystores like the Android Keystore and iOS Keychain. This ensures cryptographic isolation at the hardware level, protecting sensitive data from cold boot RAM scans and unauthorized access.

Additionally, **Securely** features a powerful custom in-app keyboard suite (**SecureTextField** + **SecureKeyboard**) that bypasses the system keyboard entirely, offering dynamic key scrambling, multiple layouts, haptic feedback, and intelligent protection modes. This suite effectively shields sensitive input from keyloggers and screen scrapers by automatically obscuring or blocking input during screen sharing attempts.

By integrating threat detection, secure storage, and protected input handling into one unified package, **Securely** eliminates the need for multiple fragmented security libraries, simplifies integration, and reduces app size overhead. This comprehensive approach helps developers building high-stakes applications—such as digital banking and healthcare—efficiently meet stringent security standards like OWASP MASVS and PCI-DSS.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1. Project Planning and Initiation

Feasibility Study (Step-by-Step)

Phase 1: Preliminary Analysis & Project Scope Definition

The primary objective of **Securely** is the creation of an all-encompassing, cross-platform Flutter package that consolidates essential security functionalities into a single solution.

Project Boundaries: The package focuses on native platform telemetry APIs for detecting debuggers, root/jailbreak status, emulators, Frida scripts, VPN usage, and active screen recording across iOS, Android, macOS, Windows, and Linux, with a prioritized emphasis on mobile platforms (iOS and Android). It also includes a platform-agnostic secure keyboard/text-field wrapper and hardware-integrated local storage for keystores.

Exclusions: Features such as automated code obfuscation, reverse-proxy network tunneling, and runtime server-side verification are considered outside the current project scope and are instead addressed by specialized infrastructure utilities.

Phase 2: Market Feasibility Analysis (or Market Research)

Looking at the current landscape on pub.dev, it’s clear that security options for Flutter are pretty fragmented. Developers currently have to juggle a mix of separate, often unmaintained libraries just to handle basic tasks like jailbreak detection, data storage, and secure input. To make matters worse, most of these tools lack support for Web or Desktop platforms and don't provide reliable protection against screen capture or recording.

Our target audience includes Flutter developers building high-stakes applications, specifically those in fields like digital banking, healthcare, or corporate VPN services where security is non-negotiable.

What makes this package different is that it’s a complete, all-in-one solution. Instead of relying on a patchwork of disconnected tools, you get a single package that combines hardware-backed encryption with a secure in-app keyboard, effectively shielding sensitive data from keyloggers, screen scrapers, and memory-based attacks.

Phase 3: Technical Feasibility Analysis

SDK and Language Viability: The technical stack involves Flutter and Dart for the core package layer, with native integrations via Swift (iOS/macOS), Kotlin (Android), and C++ (Windows/Linux).

Risks in Native Telemetry: To detect environmental threats like active Frida hooks or attached debuggers, the system must invoke low-level OS-specific commands, such as

Android root directory scans or *sysctl* calls in iOS. These operations are supported through standard Flutter Method Channels, maintaining compatibility back to Dart version 3.10.7.

Storage Implementation: Secure data persistence is achieved by routing Dart requests to the Android Keystore and iOS Keychain, utilizing AES-CBC and AES-GCM for hardware-level cryptographic isolation.

Phase 4: Financial Feasibility Analysis

As an open-source security package, securely does not generate direct SaaS revenue. Instead, financial feasibility is evaluated against development overhead vs. maintenance costs:

Development Assets: 100% open-source software tools (Flutter SDK, Xcode, Android Studio, VS Code, Git).

Maintenance Overhead: Estimated at 4 hours/week for upgrading platform dependencies and reviewing security CVEs.

Value Multiplier: Eliminates the need for expensive third-party proprietary mobile security SDKs ($10,000+/year per app) for startups and mid-market enterprises.

2. Target User Profile and Tentative Elicitation Process

Primary User Persona: Lead Mobile Security Engineers and Flutter Developers who require immediate compliance with security audits (e.g., OWASP MASVS, PCI-DSS) regarding in-app keylogging, screen capture leakage, and insecure local device storage.

Elicitation Methodology:

1. **Developer Interviews:** Conducted structured sessions with three fintech developers regarding their struggle with custom pinpad widgets closing unexpectedly or conflicting with OS screen readers.
2. **Telemetry Surveys:** Gathered data on critical attack vectors, resulting in priority scheduling for debugger detection, VPN routing confirmation, and Frida hook protection.
3. **Audit Feedback Loop:** Reconstructed security compliance checklists into functional library requirements.

3. System Requirements

Hardware Requirements: There are no unique hardware constraints for this package. Any workstation that meets standard Flutter development requirements, specifically a modern processor and sufficient RAM (minimum 8GB recommended) is fully capable of building and testing applications using this security suite.

Software Requirements: The package supports multi-platform execution and requires a minimum Flutter SDK version of 3.10.7 or higher. Compatibility is maintained across the following environments:

- **Android:** API Level 21 (Lollipop 5.0+) or higher
- **iOS:** iOS 12.0 or higher
- **macOS:** OS X 10.14 or higher
- **Windows:** Windows 10 or higher
- **Linux:** Ubuntu 20.04 or higher
- **Web:** Modern, standards-compliant browsers

4. Project Scheduling

a. Time Frame

Below is the execution schedule structured using standard Scrum sprints:

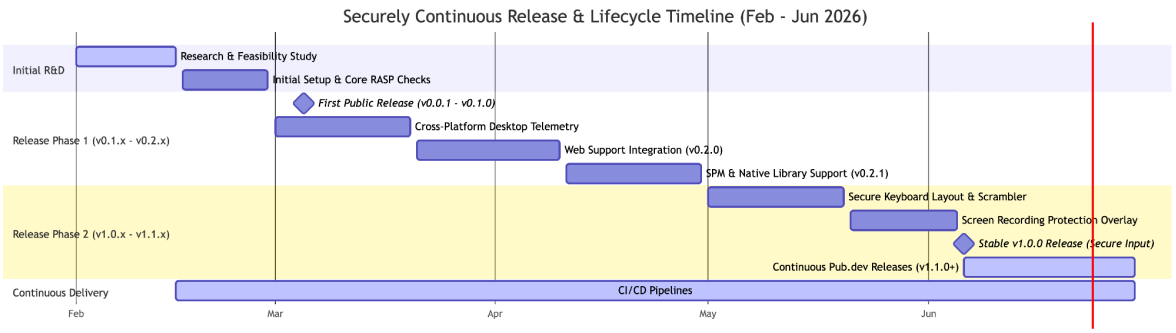

Figure-1: Gantt chart for Securely Continuous Release & Lifecycle Timeline

b. Risk Management Matrix

Security projects feature unique constraints. The table below lists predicted risks, their impact, and active mitigation steps:

Risk Category	Threat Description	Probability	Severity	Mitigation Strategy
Technical	Root bypass tools (e.g., Magisk modules) spoofing return telemetry.	High	Medium	Implement multi-indicator checks (checking files, commands, read-only path behaviors) to avoid single points of failure.

Risk Category	Threat Description	Probability	Severity	Mitigation Strategy
Operational	UI lag (>16ms per frame) during active key scrambling on low-end devices.	Medium	High	Optimize build states in SecureKeyboard by decoupling layout generation from redraw actions.
Compliance	Platform keystore updates deprecate default GCM cipher selections.	Low	Critical	Abstract algorithm configuration; allow developers to specify custom keysizes (128-bit/256-bit) and modes (GCM/CBC).
Security	Hardcoded secrets or credentials leaked in the workspace during development.	Medium	Critical	Implement automated Git prepush hooks searching for API keys and utilize isolated test configs.

CHAPTER 2: DESIGN AND IMPLEMENTATION

1. Functional Requirements

FR01	Root Detection
Description	The system must detect whether the Android device is rooted or the iOS device is jailbroken using multiple detection techniques.
Stakeholder	Developer, HostOS

FR02	Debugger Detection
Description	The system must detect if a debugger is currently attached to the running application.
Stakeholder	Developer, HostOS

FR-03	Emulator Detection
Description	The system must determine if the app is running on an emulator or simulator instead of a physical device.
Stakeholder	Developer, HostOS

FR-04	Frida Detection
Description	The system must detect the presence of Frida framework or related hooking tools.
Stakeholder	Developer, HostOS

FR-05	VPN Detection
Description	The system must detect if an active VPN connection is present on the device.
Stakeholder	Developer, HostOS

FR-06	Screen Recording Detection
Description	The system must detect active screen recording or screen sharing and notify the app in real-time.
Stakeholder	Developer, HostOS

FR-07	Screenshot Detection
Description	The system must detect when a screenshot is taken and trigger an event.
Stakeholder	Developer, HostOS

FR-08	Write Data
Description	The system must support writing data to secure storage.
Stakeholder	Developer, HostOS

FR-09	Read Data
Description	The system must support reading data from secure storage.
Stakeholder	Developer, HostOS

FR-10	Delete Data
Description	The system must support deleting specific data entries from secure storage.
Stakeholder	Developer, HostOS

FR-11	Check Data
Description	The system must support verifying if a data entry exists in secure storage.
Stakeholder	Developer, HostOS

FR-12	Clear All
Description	The system must support clearing all data from secure storage.
Stakeholder	Developer, HostOS

FR-13	Change Algorithm
Description	The system must support configuring the encryption algorithm (e.g., AES-GCM, AES-CBC).
Stakeholder	Developer

FR-14	Change Key Size
Description	The system must support configuring the cryptographic key size.
Stakeholder	Developer

FR-15	Change Keyboard Type
Description	The system must support switching between different keyboard layouts (e.g., numeric, alphanumeric).
Stakeholder	Developer, User

FR-16	Change between Bottom Sheet or Inline
Description	The system must support rendering the secure keyboard as either a bottom sheet or an inline widget.
Stakeholder	Developer, User

FR-17	Keyboard Key Scrambling
Description	The secure keyboard must support key position randomization (once or after every tap).
Stakeholder	Developer, User

FR-18	Screen Share Protection
Description	When screen recording is detected, the keyboard must support obscuring labels or blocking input with a security shield.
Stakeholder	Developer, HostOS

FR-19	Real-time Security Events
Description	The system must provide Dart streams for screenshot and screen recording events.
Stakeholder	Developer, HostOS

FR-20	Developer Mode Detection
Description	The system must detect if Developer Mode is enabled on the device.
Stakeholder	Developer, HostOS

FR-21	USB Debugging Detection
Description	The system must detect if USB Debugging is active on the device.
Stakeholder	Developer, HostOS

2. Non-Functional Requirements

Security: All storage operations must use platform-native secure buffers that zero out parameters post-operation to resist cold boot RAM scans.

Performance: Interactive soft keyboard buttons must respond to taps in under 16ms to target a fluid 60 FPS standard.

Maintainability: The codebase must adhere to modular design patterns to support long-term extensibility. All public methods require comprehensive documentation, and a minimum of 80% unit test coverage must be maintained to facilitate safe code refactoring.

Compatibility: The codebase must compile across iOS, Android, macOS, Linux, Windows, and Web.

3. Object-oriented System Design using UML

a. Use Case Diagram

Below is the system boundary mapping demonstrating how developer, user and HostOS interact with securely:

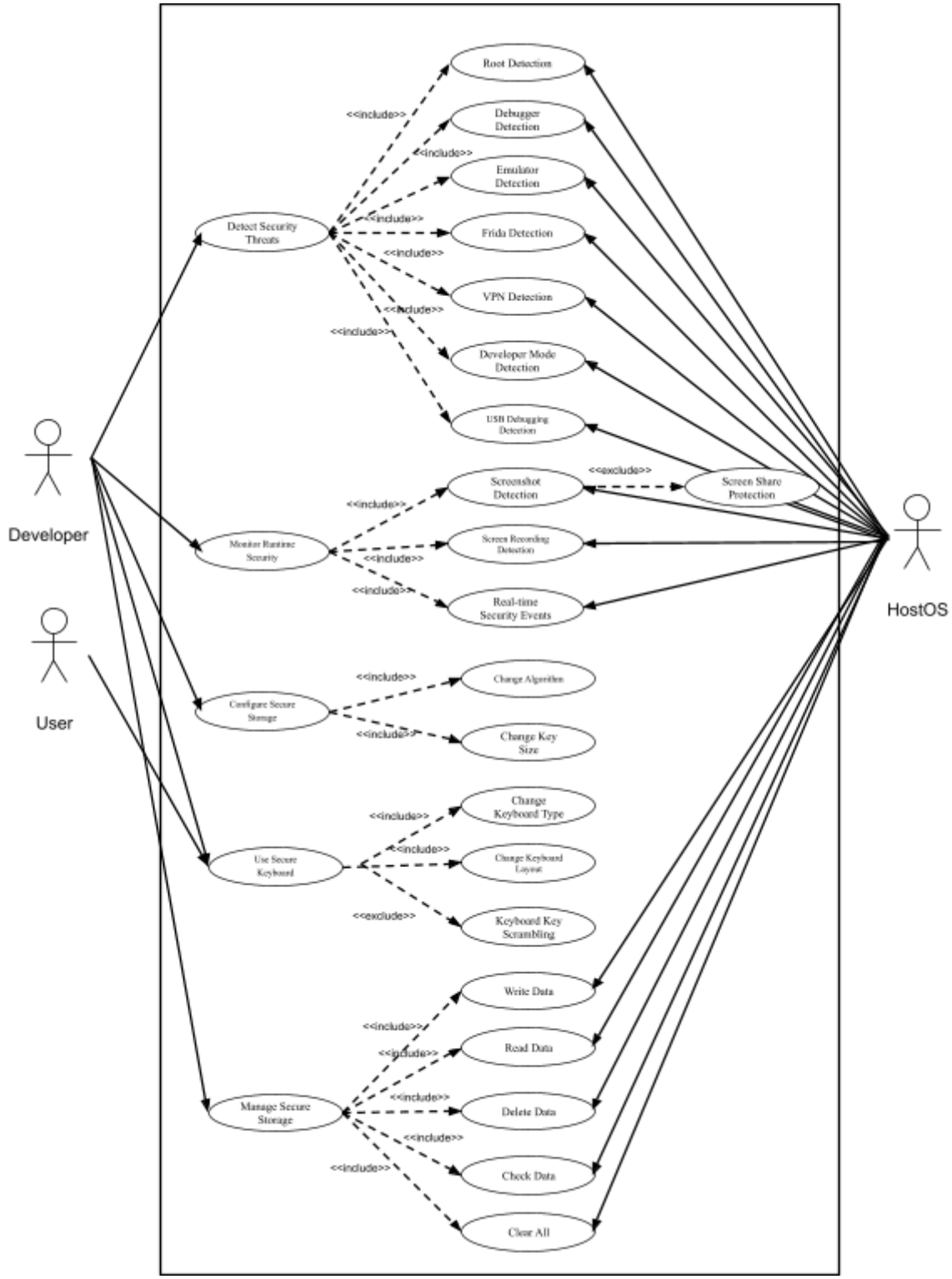

Figure-2: Use case diagram

b. Use Case Descriptions

Category 1: Detect Security Threats (RASP)

Use Case 1.1: Root/Jailbreak Detection

Use Case Name	Root/Jailbreak Detection
Description	Scans the device environment for artifacts, configuration files, and permissions indicating the device has been rooted (Android) or jailbroken (iOS).
Preconditions	The host application is initialized and has loaded the securely plugin.
Success End Condition	The API successfully determines the root/jailbreak status and returns a boolean value (true for rooted, false for secure) to the host application.
Failed End Condition	The query fails or is bypassed by masking tools, returning an incorrect security status or throwing a platform exception.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS File System, Su binaries
Trigger	The host application calls Securely.isRootDetected().
Main Success Scenario	1. Host application requests root detection status. 2. Dart plugin invokes the native Method Channel isRootDetected. 3. Android/iOS native code runs checks (e.g., checking for su binary, busybox, test-keys, Cydia/Sileo files, or sandbox writing privileges). 4. Native code returns the boolean status. 5. Dart plugin returns the result to the caller.
Alternative Flows	**Alternative Flow A (Execution Exception):** If native checks fail due to runtime permission changes, the platform channel catches the error and returns a default safe value (true to assume threat/unsecure).
Quality Requirements	Execution time must be under 50ms, with zero background battery usage when idle.

Use Case 1.2: Debugger Detection

Use Case Name	Debugger Detection
Description	Inspects OS process flags to verify if a debugger (e.g., LLDB, GDB, JDWP) is attached to the running application process.
Preconditions	The application is running.
Success End Condition	The application receives a boolean indicating whether a debugger is attached.
Failed End Condition	Anti-debugging bypass hooks intercept the system query, hiding the debugger.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS Debugging Flags / APIs
Trigger	The host application calls Securely.isDebuggerDetected().
Main Success Scenario	1. Host application queries debugging status. 2. Native Kotlin code executes android.os.Debug.isDebuggerConnected() (Android) or native Swift code checks sysctl with the P_TRACED flag (iOS). 3. The native layer returns the status. 4. Dart resolves the future boolean.
Alternative Flows	**Alternative Flow A (Spoofed Environment):** If system calls are intercepted, fallback secondary scans (like checking if port 8600 is bound) are used to confirm debugging.
Quality Requirements	Checks must run in under 10ms to prevent lag during critical security checks.

Use Case 1.3: Emulator Detection

Use Case Name	Emulator Detection
Description	Checks device hardware properties (model, manufacturer, build fingerprint) to identify if the app is running on a virtualized emulator or simulator.
Preconditions	The application is active.
Success End Condition	Emulated hardware is identified and reported as a boolean.
Failed End Condition	Advanced emulator hides hardware identifiers, bypassing checks.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS System Hardware Info
Trigger	The host application calls Securely.isEmulatorDetected().
Main Success Scenario	1. Host application requests environment status. 2. Native code reads system constants (e.g., Build.FINGERPRINT, Build.MODEL, Build.HARDWARE on Android; or checking target environments on iOS). 3. The native code evaluates if they contain virtual keywords (e.g., sdk_gphone, simulator, vbox86). 4. The system returns true if an emulator is detected.
Alternative Flows	**Alternative Flow A (Desktop/Web Platform):** If executed on a desktop or browser, returns false by default or flags as simulated depending on host parameters.
Quality Requirements	Evaluates a minimum of 5 distinct hardware indicators for verification accuracy.

Use Case 1.4: Frida Detection

Use Case Name	Frida Hook Detection
Description	Searches memory spaces, running threads, and port mappings to check for dynamic binary instrumentation tools (Frida) injecting hooks into the app.
Preconditions	The application is active on a mobile platform.
Success End Condition	Frida injection signals are recognized, and the API returns true.
Failed End Condition	Frida hides its dynamic library names and overrides port bindings, bypassing detection.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS Memory and Process management
Trigger	The host application calls Securely.isFridaDetected().
Main Success Scenario	1. Host application requests Frida hook status. 2. Native C/Kotlin/Swift code scans list of loaded libraries for strings containing frida. 3. Native code attempts to bind to default Frida socket ports (e.g., 27042) to check if they are occupied. 4. The presence status is returned.
Alternative Flows	**Alternative Flow A (Memory read blocked):** If system security restrictions prevent library scanning, the system raises a fallback check on files under /data/local/tmp.
Quality Requirements	Memory queries must run asynchronously to maintain a 60fps frame rate.

Use Case 1.5: VPN Detection

Use Case Name	VPN Detection
Description	Queries the current network interface configurations to confirm if network traffic is actively routed through a VPN.
Preconditions	A network connection is active on the device.
Success End Condition	The network interfaces list successfully identifies a VPN transport type or tunnel interface.
Failed End Condition	Query to network capabilities returns null or fails.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS Network Interfaces
Trigger	The host application calls Securely.isVpnDetected().
Main Success Scenario	1. Host application requests network check. 2. Native Android ConnectivityManager (or iOS NEVPNManager/CFNetwork) queries active routing. 3. If interfaces named tun0, ppp0, or transport types matching TRANSPORT_VPN are active, flags as VPN. 4. Returns boolean status to Dart.
Alternative Flows	**Alternative Flow A (Offline Status):** If no network interfaces are found, the API resolves to false immediately.
Quality Requirements	API must complete network interface scanning in under 30ms.

Use Case 1.6: Developer Mode Detection

Use Case Name	Developer Mode Detection
Description	Identifies whether the system "Developer Options/Mode" settings are enabled on the device.
Preconditions	System settings are accessible.
Success End Condition	The status is accurately determined and returned.
Failed End Condition	Platform settings queries are blocked by restricted profiles.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS Settings Provider
Trigger	The host application calls Securely.isDeveloperModeDetected().
Main Success Scenario	1. Host application queries developer options. 2. Native code calls system content resolver checking Settings.Global.development_settings_enabled (Android) or profiles (iOS). 3. The result returned to Dart.
Alternative Flows	**Alternative Flow A (iOS Platform):** Because iOS does not provide a direct global setting API, the system checks signature profile modes or debugger hook traces.
Quality Requirements	Must read settings database using cached queries to avoid file system read delays.

Use Case 1.7: USB Debugging Detection

Use Case Name	USB Debugging Detection
Description	Verifies if Android ADB (Android Debug Bridge) or iOS USB debugging is actively connected and allowed over USB interface.
Preconditions	The application is running on a supported mobile device.
Success End Condition	App detects active ADB setting state and returns boolean.
Failed End Condition	Settings query fails or is blocked.
Primary Actor	Developer (Host Application)
Secondary Actor	Android ADB Service
Trigger	The host application calls Securely.isUsbDebuggingDetected().
Main Success Scenario	1. Host application queries USB Debugging state. 2. Android native layer checks if Settings.Global.adb_enabled is set to 1. 3. Returns boolean status back to Dart.
Alternative Flows	**Alternative Flow A (iOS/Desktop):** Immediately returns false (or queries USB transfer bindings if applicable).
Quality Requirements	Query execution latency must be under 5ms.

Category 2: Monitor Runtime Security

Use Case 2.1: Screenshot Detection

Use Case Name	Screenshot Detection
Description	Registers a listener that intercepts user screenshots and triggers security callbacks inside the app.
Preconditions	The application is running in the foreground.
Success End Condition	The user takes a screenshot, and an event stream notification is emitted.
Failed End Condition	The user captures a screenshot, but the observer fails to detect the media or notification event.
Primary Actor	EndUser
Secondary Actor	Host OS Photo Library Observer / Screen Notifications
Trigger	EndUser performs a screenshot keystroke combination.
Main Success Scenario	1. EndUser takes a screenshot of the app. 2. Native layer catches the screenshot notification (UIApplicationUserDidTakeScreenshotNotification on iOS) or detects file write in the media storage (Android). 3. Native code sends an event across EventChannel('securely/onScreenshot'). 4. The host application handles the event (e.g., logs out user, flags transaction, clears cache).
Alternative Flows	**Alternative Flow A (Access Denied to Storage):** On Android, if storage permissions are restricted, screenshot detection falls back to lifecycle focus monitoring or flags warning.
Quality Requirements	The alert stream must deliver the event within 150ms of the screenshot capture.

Use Case 2.2: Screen Recording Detection

Use Case Name	Screen Recording Detection
Description	Queries and monitors whether the system screen recording or screen mirror daemon is active.
Preconditions	The app is in the foreground.
Success End Condition	An active screen recording status is detected and reports updates.
Failed End Condition	A recording is running, but the package returns false.
Primary Actor	HostOS / EndUser
Secondary Actor	Screen Capture System Service
Trigger	Screen recording starts or stops.
Main Success Scenario	1. EndUser starts recording their screen via the OS control panel. 2. Native layer identifies active capture (e.g., UIScreen.main.isCaptured on iOS, virtual display count on Android). 3. The native layer emits status true to Dart via onScreenRecordingChanged. 4. The host application hides sensitive layout components.
Alternative Flows	**Alternative Flow A (Startup Check):** On app launch, Securely.isScreenRecordingDetected() checks initial status.
Quality Requirements	State change must propagate to Dart in less than 300ms.

Use Case 2.3: Screen Share Protection

Use Case Name	Intercept Screen Share & Block Input
Description	Obscures the custom keyboard layout or overlays an input blocker when screen mirroring/recording is detected.
Preconditions	SecureTextField / SecureKeyboard is focused and visible.
Success End Condition	Keyboard content is hidden or keyboard interaction is blocked when casting/recording.
Failed End Condition	Screen share captures plain-text keyboard buttons or PIN codes.
Primary Actor	EndUser (Screen Share initiator)
Secondary Actor	SecureKeyboard Widget
Trigger	The screen recording status shifts to active while the secure keyboard is open.
Main Success Scenario	1. Screen mirroring begins. 2. SecureKeyboard receives a state change callback. 3. If obscureMode is set to blockKeyboard, a blocker screen overlaying the keys displays: "SECURE INPUT BLOCKED". Taps are rejected. 4. If set to obscureLabels, labels change to padlocks, but entry remains allowed. 5. mirroring stops: keyboard layout is restored.
Alternative Flows	**Alternative Flow A (Disable Keyboard Option):** If disableKeyboardDuringRecording is enabled, the keyboard automatically closes and refuses to open.
Quality Requirements	Protective overlay must render inside 1 frame (~16ms) to prevent visual leakage.

Use Case 2.4: Real-time Security Events

Use Case Name	Real-time Security Events Stream
Description	Publishes immediate notifications of security violations (screenshots, recording, platform modifications) to host Dart application listeners.
Preconditions	The app has active listeners on RASP streams.
Success End Condition	Stream events are delivered containing specific violation data.
Failed End Condition	Stream drops connections silently.
Primary Actor	Developer (Host Application)
Secondary Actor	Host OS / Security Monitors
Trigger	A security threat state changes (e.g., VPN toggled, screenshot taken).
Main Success Scenario	1. Developer initializes Securely.onScreenRecordingChanged.listen(...). 2. An event triggers on the OS. 3. Event channel pushes serialization map. 4. The host application acts on the message.
Alternative Flows	**Alternative Flow A (No listeners):** Events are buffered or discarded until a listener registers.
Quality Requirements	Event channels must ensure delivery ordering and complete callbacks under 100ms.

Category 3: Manage Secure Storage

Use Case 3.1: Write Data

Use Case Name	Write Secure Data
Description	Encrypts a key-value payload and writes it to the platform's hardware-backed cryptographic storage.
Preconditions	StoreSecurely is initialized.
Success End Condition	Ciphertext is saved successfully in Keychain/Keystore.
Failed End Condition	Write fails due to full storage space, locking, or cryptographic mismatch.
Primary Actor	Developer (Host Application)
Secondary Actor	Native iOS Keychain / Android Keystore
Trigger	The host application calls StoreSecurely.write(key, value).
Main Success Scenario	1. Host application calls write("session_token", "jwt123"). 2. StoreSecurely checks encryption parameters. 3. Key-value pair sent via Method Channel. 4. Native layer encrypts value using hardware key and saves ciphertext. 5. Native returns true.
Alternative Flows	**Alternative Flow A (Overwrite):** If key exists, native overwrites current ciphertext with new payload.
Quality Requirements	Write transactions must complete in less than 150ms.

Use Case 3.2: Read Data

Use Case Name	Read Secure Data
Description	Locates, decrypts, and returns the plaintext value of a stored key.
Preconditions	Key was previously written.
Success End Condition	The plaintext string is returned to the host application.
Failed End Condition	Decryption fails (key changed/data corrupted) or key does not exist.
Primary Actor	Developer (Host Application)
Secondary Actor	Native iOS Keychain / Android Keystore
Trigger	The host application calls StoreSecurely.read(key).
Main Success Scenario	1. Host application calls read("session_token"). 2. Native code loads ciphertext, queries hardware security module for decryption key, decrypts payload. 3. Native returns a plaintext string.
Alternative Flows	**Alternative Flow A (Key not found):** Returns null without throwing exceptions.
Quality Requirements	Read latency must be under 100ms.

Use Case 3.3: Delete Data

Use Case Name	Delete Secure Key
Description	Deletes a specific key and its ciphertext from the hardware secure storage.
Preconditions	Key exists.
Success End Condition	Entry is removed; next reads return null.
Failed End Condition	Entry deletion is blocked by OS locking permissions.
Primary Actor	Developer (Host Application)
Secondary Actor	Native iOS Keychain / Android Keystore
Trigger	The host application calls StoreSecurely.delete(key).
Main Success Scenario	1. The host application calls delete("session_token"). 2. Native code deletes the alias entry. 3. Returns true.
Alternative Flows	**Alternative Flow A (Key does not exist):** Returns false or completes silently.
Quality Requirements	Erased sector data must be wiped immediately.

Use Case 3.4: Check Data

Use Case Name	Check Key Presence
Description	Verifies if a specific key exists in secure storage without performing decryption.
Preconditions	Secure storage is initialized.
Success End Condition	Returns boolean indicating presence.
Failed End Condition	System query fails.
Primary Actor	Developer (Host Application)
Secondary Actor	Native Keychain/Keystore
Trigger	The host application calls StoreSecurely.containsKey(key).
Main Success Scenario	1. Host application checks key containsKey("session_token"). 2. Native code checks for alias presence. 3. Returns status boolean.
Alternative Flows	**Alternative Flow A (Device Locked):** If hardware is locked, queries may return false or wait depending on policy flags.
Quality Requirements	The query completes in under 20ms.

Use Case 3.5: Clear All

Use Case Name	Clear Secure Storage
Description	Deletes all secure keys and ciphertext data managed by the package.
Preconditions	Entries exist in the package storage domain.
Success End Condition	All package-managed keys are purged.
Failed End Condition	Clear operation fails halfway, leaving orphaned entries.
Primary Actor	Developer (Host Application)
Secondary Actor	Native Keychain/Keystore
Trigger	The host application calls StoreSecurely.clear().
Main Success Scenario	1. Host application requests complete storage wipe. 2. Native layer iterates through stored package aliases and purges them. 3. Returns true on completion.
Alternative Flows	**Alternative Flow A (Empty Storage):** Returns true immediately.
Quality Requirements	Guarantees removal of all stored records.

Category 4: Configure Secure Storage

Use Case 4.1: Change Algorithm

Use Case Name	Change Storage Algorithm
Description	Switches the cryptographic cipher algorithm used for writing secure data.
Preconditions	The host app has instantiated StoreSecurely.
Success End Condition	Storage configurations are updated, and subsequent writes apply the new cipher.
Failed End Condition	Selecting an unsupported algorithm defaults to a standard cipher or fails.
Primary Actor	Developer (Host Application)
Secondary Actor	None
Trigger	The developer calls StoreSecurely.setAlgorithm(SecurelyAlgorithm.aesCbc).
Main Success Scenario	1. The developer changes configuration mode to AES-CBC. 2. Future write payloads include the algorithm tag. 3. Native layer uses the matching native encryption protocol.
Alternative Flows	**Alternative Flow A (Read Isolation):** Reading a key previously encrypted under AES-GCM when the current setting is AES-CBC resolves to null to prevent block decryption errors.
Quality Requirements	Configuration changes must take effect immediately inside the Dart instance.

Use Case 4.2: Change Key Size

Use Case Name	Change Cryptographic Key Size
Description	Configures the key length used by cryptographic keys (e.g. 128-bit vs 256-bit).
Preconditions	Instantiated StoreSecurely.
Success End Condition	Future write requests generate keys of the specified length.
Failed End Condition	Selecting unsupported sizes throws a compile-time or runtime exception.
Primary Actor	Developer (Host Application)
Secondary Actor	None
Trigger	The developer calls StoreSecurely.setKeySize(SecurelyKeySize.bits128).
Main Success Scenario	1. The developer switches the key size parameter to 128 bits. 2. Native layer generates AES-128 keys for subsequent writes.
Alternative Flows	**Alternative Flow A (Isolation):** Reads to 256-bit keys while set to 128-bit config return null to avoid cipher key mismatch.
Quality Requirements	Must support at least standard 128 and 256-bit cryptographic sizes.

Category 5: Use Secure Keyboard

Use Case 5.1: Use Secure Keyboard

Use Case Name	Input Passcode via Secure Keyboard
Description	Suppresses system keyboard overlays and displays a custom in-app keyboard to secure user text fields.
Preconditions	SecureTextField is loaded and focused by the user.
Success End Condition	Secure keyboard renders, accepts touches, inputs text, and dismisses on Done.
Failed End Condition	The native OS soft keyboard displays, or custom keyboard crashes.
Primary Actor	EndUser
Secondary Actor	SecureTextField Widget
Trigger	EndUser taps the secure input field.
Main Success Scenario	1. EndUser focuses on the text field. 2. The widget suppresses native OS input focus. 3. Keyboard bottom sheet slides into view. 4. EndUser types characters. 5. Characters are loaded directly into the controller buffer. 6. EndUser taps "Done". 7. Sheet dismisses and text field unfocuses.
Alternative Flows	**Alternative Flow A (Inline Render):** Renders inline directly within the view layout instead of rising as a bottom sheet.
Quality Requirements	Bottom-sheet slider animation must render smoothly under 16ms (60fps).

Use Case 5.2: Change Keyboard Type

Use Case Name	Change Keyboard Input Type
Description	Alters the secure keyboard layout structure between standard numeric pad or alphanumeric layout.
Preconditions	SecureTextField configuration is updated.
Success End Condition	The keyboard builds the layout matching specified type.
Failed End Condition	System defaults to numeric despite alphanumeric config.
Primary Actor	Developer (Host Application)
Secondary Actor	EndUser (interacts with correct layouts)
Trigger	Developer configures keyboardType: SecureKeyboardType.alphanumeric.
Main Success Scenario	1. The developer specifies the alphanumeric keyboard type. 2. EndUser focuses on the field. 3. The keyboard widget builds an alphanumeric key list (letters, shift keys, symbol toggles). 4. EndUser inputs string characters.
Alternative Flows	**Alternative Flow A (Numeric Type):** Numeric configurations construct simple 0-9 layouts.
Quality Requirements	Layout coordinates must adjust responsively to fit various screen sizes (phones and tablets).

Use Case 5.3: Change Keyboard Layout

Use Case Name	Change Keyboard Layout Theme
Description	Restyles colors, heights, padding, border radii, and text fonts of the custom keyboard buttons and background headers.
Preconditions	SecureKeyboardTheme is defined.
Success End Condition	Custom styles are painted onto the keyboard canvas.
Failed End Condition	Visual bugs, overlapping letters, or default layout fallback occurs.
Primary Actor	Developer (Host Application)
Secondary Actor	None
Trigger	The developer defines and passes a custom SecureKeyboardTheme object.
Main Success Scenario	1. The developer specifies customized colors and height properties. 2. During widget paint, keyboard buttons read values from the theme. 3. Styled buttons are rendered to EndUser.
Alternative Flows	**Alternative Flow A (No Theme Specified):** Renders default high-contrast dark or light theme according to system preferences.
Quality Requirements	Dynamic theme updates must compile cleanly without triggering layout cycles or performance leaks.

Use Case 5.4: Keyboard Key Scrambling

Use Case Name	Scramble Keyboard Keys
Description	Randomizes the button positions on the custom keyboard grid to block layout tracking malware and shoulder surfing.
Preconditions	Scrambling mode is active (once or always).
Success End Condition	Grid keys are drawn in scrambled positions.
Failed End Condition	Keys remain static or fail to map values to correct buttons.
Primary Actor	EndUser
Secondary Actor	SecureKeyboard Widget
Trigger	The secure keyboard opens or a key is pressed while scramble type is enabled.
Main Success Scenario	1. EndUser opens the keyboard with scramble mode set to always. 2. The grid array is shuffled randomly using Dart's random utilities. 3. Keyboard renders buttons in randomized layout coordinates. 4. EndUser taps the button; character is entered. 5. The grid reshuffles key locations instantly.
Alternative Flows	**Alternative Flow A (Scramble Once):** Shuffles layout once upon opening and remains static until dismissed.
Quality Requirements	The shuffling algorithm must complete in less than 1ms.

c. Activity Diagrams

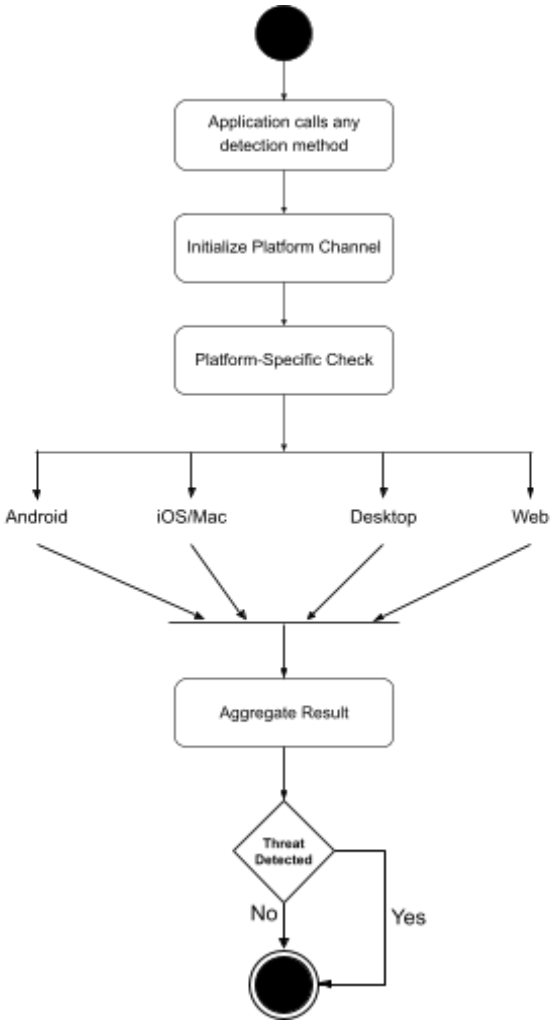

Figure-3: Activity Diagram For All Detection Method

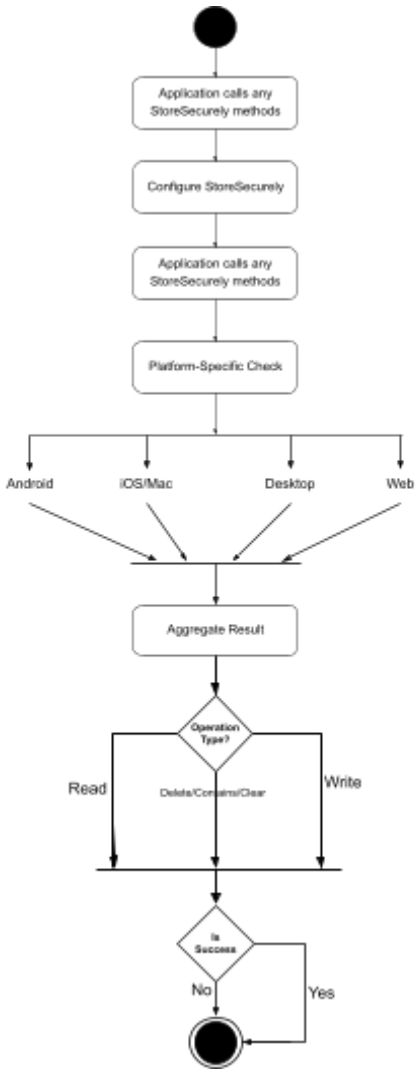

Figure-4: Activity Diagram For StoreSecurely

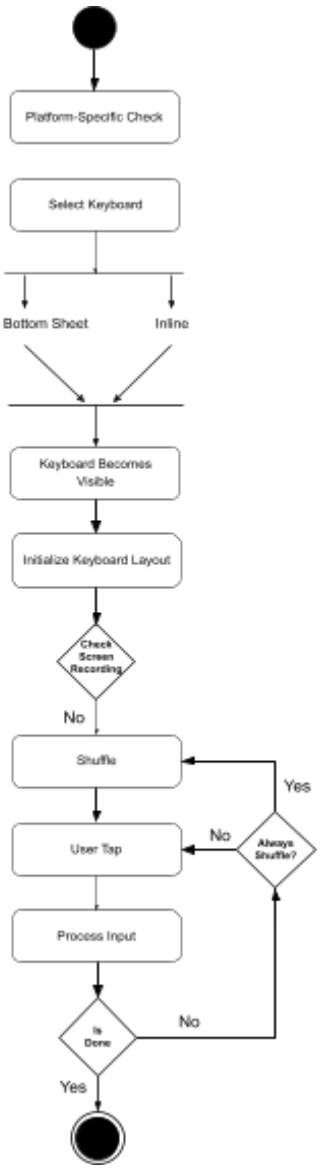

Figure-5: Activity Diagram For Secure Text Field and Secure Keyboard

d. Sequence Diagram

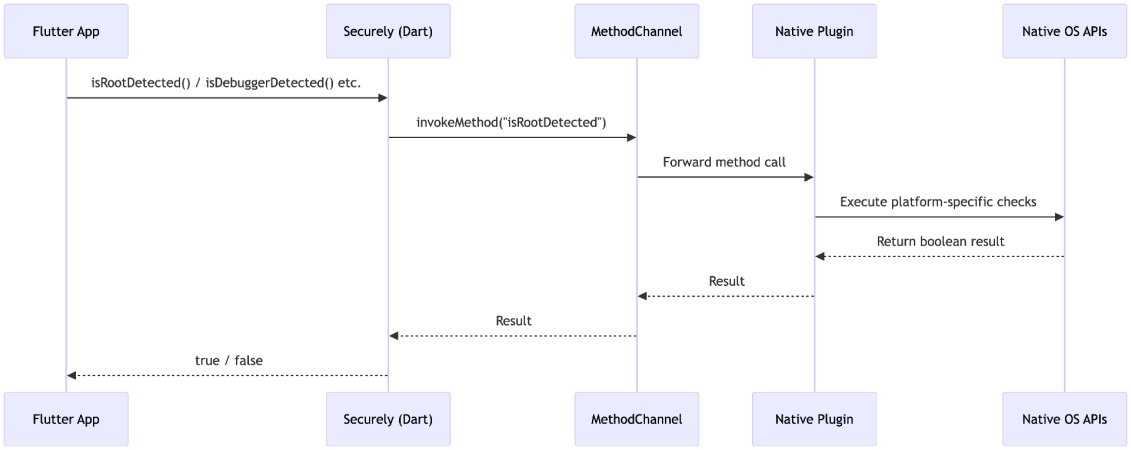

Figure-6: Sequence Diagram For All Detection Methods

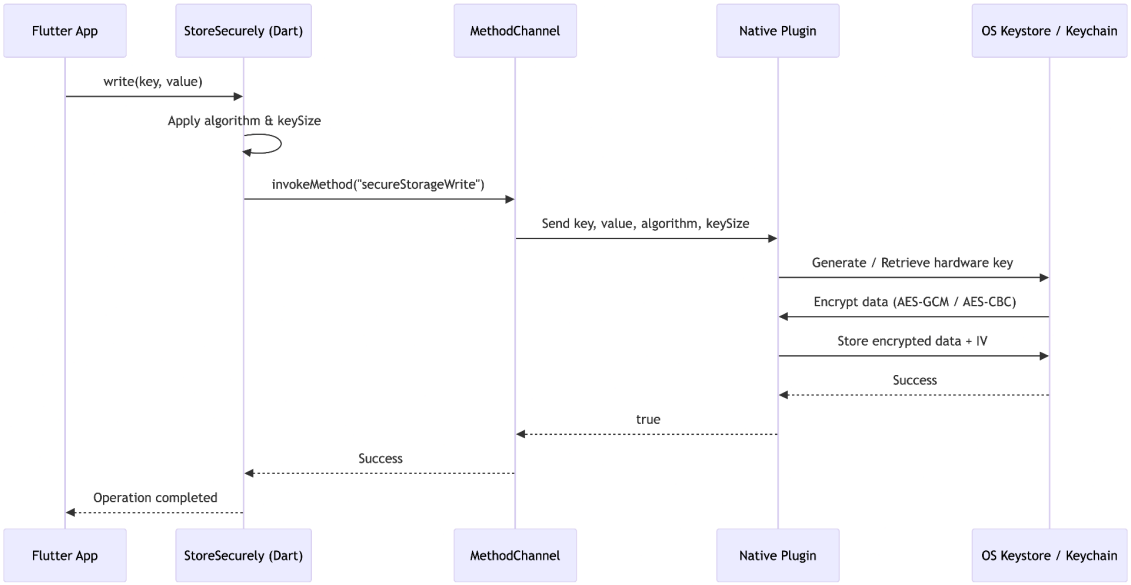

Figure-7: Sequence Diagram For StoreSecurely

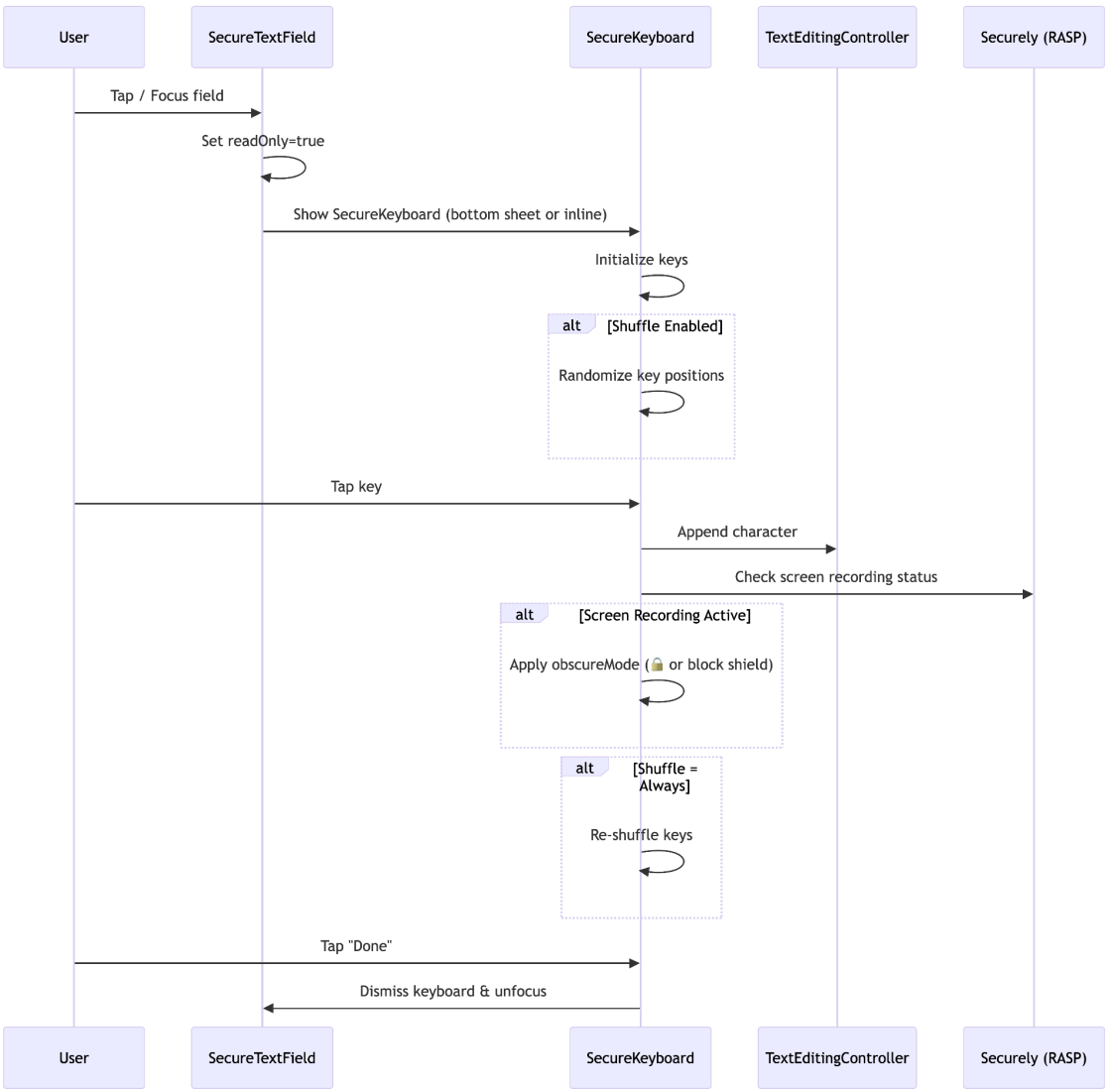

Figure-8: Sequence Diagram For Secure Text Field and Secure Keyboard

e. Class Diagram

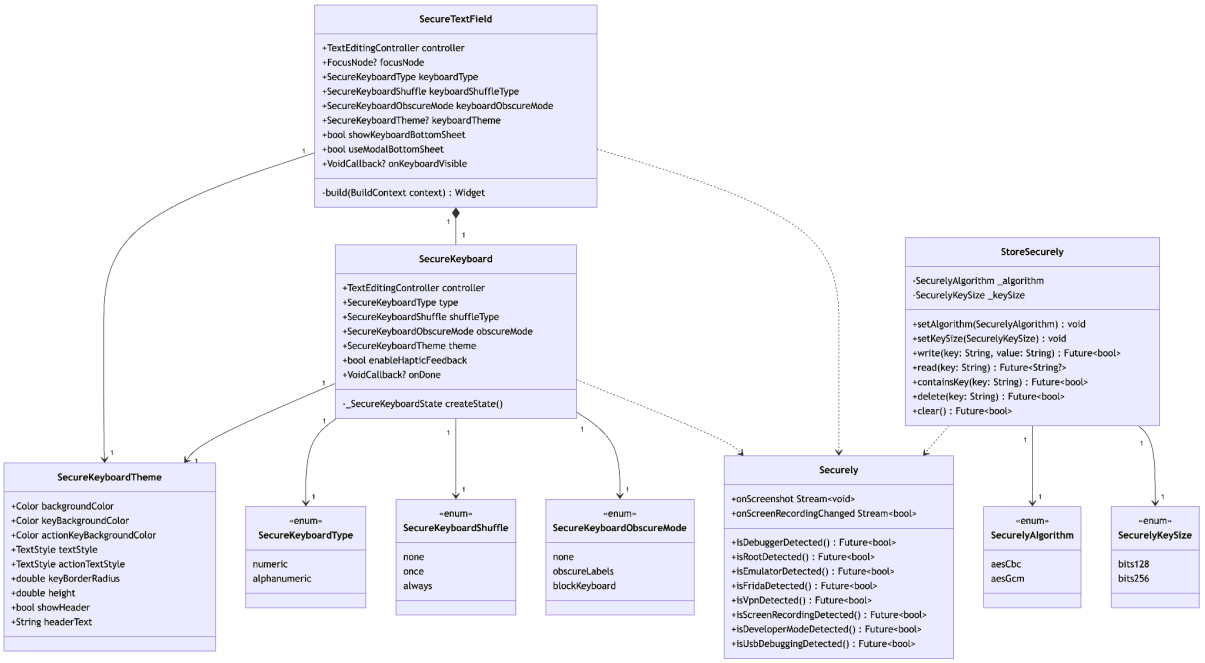

Figure-9: Class Diagram For Securely

4. Coding Architecture

Platform Channels: Communication uses standard MethodChannel('securely') for execution-on-demand telemetry, and EventChannel for streaming telemetry events.

Web Fallback: The web runtime overrides native calls using securely_web.dart, writing data encrypted using web encryption modules or plain fallbacks to browser local storage.

Widget Structure: UI components use standard Flutter styling tokens allowing total layout configuration.

CHAPTER 3: SOFTWARE TESTING

1. Testing Features

securely features high-testability layout hooks:

Keypad Value Insertion: Verifies that tapping buttons modifies the text buffer at the cursor.

Key Shuffling Output: Compares randomized array orders against standard lists.

Encryption Isolation: Confirms that altering storage configuration parameters (e.g. changing keysize from 256-bit to 128-bit) correctly isolates keys.

Recording Event Dispatch: Mocks method channel bindings to test UI reaction to screen capture events.

2. Testing Strategies

Mock Method Channels: Simulates platform RASP returns in host Dart files to execute logic pathways without requiring running Android/iOS hardware.

Widget Tester Framework: Employs Flutter Widget Tests to inject simulated gestures (tap, drag, focus) on the soft keyboard buttons.

Integration Tests: Validates target runtime environments (Android emulators, iOS simulators) using the testbench application in example/.

3. System Testing (Test Cases and Results Matrix)

These tests reflect actual test executions configured in test/securely_test.dart and test/secure_keyboard_test.dart:

Test ID	Feature Under Test	Input / Action	Expected Result	Actual Result	Status
TC-101	isDebuggerDetected	Query method invocation	Returns a boolean value (true/false)	Returned false (on test stub)	✅ PASS
TC-102	isRootDetected	Query method invocation	Returns a boolean value (true/false)	Returned false (on test stub)	✅ PASS
TC-103	StoreSecurely write/read	Write "hello_world" on test_key	Reading key returns "hello_world"	Returned "hello_world"	✅ PASS

Test ID	Feature Under Test	Input / Action	Expected Result	Actual Result	Status
TC-104	StoreSecurely cipher isolation	Write "hello_cbc" in AES-128. Switch to AES-256.	Reading test_key under AES-256 returns null.	Returned null (correctly isolated)	PASS
TC-105	SecureKeyboard Numeric digits	Tap "1", then "5" on keyboard	Controller string reads "15"	Controller read "15"	PASS
TC-106	SecureKeyboard Backspace	Tap "1", "5", then "⌫"	Controller string reads "1"	Controller read "1"	PASS
TC-107	SecureKeyboard Clear	Tap "1", "5", then "Clear"	Controller string is empty	Controller is empty	PASS
TC-108	Alphanumeric Shift	Tap Shift key "⇧"	Keyboard displays capitalized letters	Displayed capitals	PASS
TC-109	Alphanumeric Symbols	Tap Symbol key "?123"	Keypad switches to symbols layout	Symbols loaded	PASS
TC-110	SecureTextField System Block	Focus input field	Read-only set to true, system keyboard blocked	Native keyboard blocked	PASS
TC-111	Bottom Sheet Dismissal	Tap "Done" or barrier outside sheet	Bottom sheet slides away, textfield loses focus	Dismissed successfully	PASS

CHAPTER 4: DEPLOYMENT AND MAINTENANCE

1. Agile Methodology Sprint Cycles

The project applies Agile principles structured around 1-week Sprints:

Sprint Backlog Management: Tasks are organized by security risk severity (e.g. storage leaks prioritized over visual theme enhancements).

Review & Retrospectives: End-of-sprint reviews trace regression bugs in UI layout shifts across target platforms.

CI/CD Pipeline: Active GitHub Actions build and verify code standards using flutter analyze and flutter test upon every push.

2. Software Release Life Cycle (SRLC)

The release of securely conforms to structured versioning milestones:

Stage	Status	Version
Pre-Alpha	Completed	v0.0.2
Alpha	Completed	v0.0.4
Beta	Completed	v0.1.0
Release Candidate	Completed	v0.2.3
Stable Release	**Current**	**v1.1.0**
Maintenance & Patching	Ongoing	v1.1.x

Stable Deployment: Deployed onto the official package repository, pub.dev, using clean semantic versioning (v1.1.0).

Maintenance Protocol: Bi-weekly scanning of Android/iOS security advisories. Platform dependencies are updated to mitigate vulnerabilities in upstream channels.

CHAPTER 5: USER MANUAL

1. Integration Setup

Add the package to your Flutter project's pubspec.yaml dependencies:

```
dependencies:
  securely: ^1.1.0
```

Ensure platform configurations are set up:

iOS/macOS: Verify sandbox keychain sharing permissions are configured inside Xcode.

Android: Ensure minimum SDK version is set to 21 inside android/app/build.gradle.

2. Code Examples & Customization

Scenario A: Telemetry Checks & RASP

Initialize early checks to intercept execution if threat vectors exist:

```
import 'package:securely/securely.dart';

void runSecurityCheck() async {
  bool isDebugger = await Securely.isDebuggerDetected();
  bool isRooted   = await Securely.isRootDetected();
  bool isRecording = await Securely.isScreenRecordingDetected();

  if (isDebugger || isRooted) {
    // Intercept threat: Terminate session or warn user
    print("Application execution halted due to RASP threats.");
  }
}
```

Scenario B: SecureTextField & SecureKeyboard

Instantiate a styled SecureTextField with randomized key arrays:

```
SecureTextField(
  controller: _pinController,
  obscureText: true,
  keyboardType: SecureKeyboardType.numeric,
  keyboardShuffleType: SecureKeyboardShuffle.always,
  keyboardObscureMode: SecureKeyboardObscureMode.blockKeyboard,
  keyboardTheme: SecureKeyboardTheme(
    backgroundColor: const Color(0xFF10101C),
    keyBackgroundColor: const Color(0xFF1A1A2B),
    actionKeyBackgroundColor: const Color(0xFF24243D),
    textStyle: const TextStyle(fontSize: 22, color: Colors.white),
    actionTextStyle: const TextStyle(fontSize: 15, color: Colors.cyanAccent),
    height: 340.0,
  ),
  decoration: const InputDecoration(
    labelText: 'Enter Secure PIN',
    border: OutlineInputBorder(),
  ),
)
```

Scenario C: StoreSecurely

Configure and use secure storage to safely persist, retrieve, and delete sensitive credentials:

```
import 'package:securely/securely.dart';

void manageCredentials() async {
  final storage = StoreSecurely();

  // Configure encryption options (optional: defaults to GCM / 256-bit)
  storage.setAlgorithm(SecurelyAlgorithm.aesCbc);
  storage.setKeySize(SecurelyKeySize.bits256);

  // Write a secure key-value pair
  await storage.write(key: 'session_token', value: 'jwt_secure_token_123');

  // Check if key exists
  bool hasToken = await storage.containsKey(key: 'session_token');
  if (hasToken) {
    // Read the secure plaintext value
    String? token = await storage.read(key: 'session_token');
    print('Retrieved token: $token');
  }

  // Delete key-value pair
```

```
  await storage.delete(key: 'session_token');

  // Clear all package-managed keys
  await storage.clear();
}
```

3. App Screenshots

Please run the example/ app and replace the links below with screenshots taken from your mobile device/emulator:

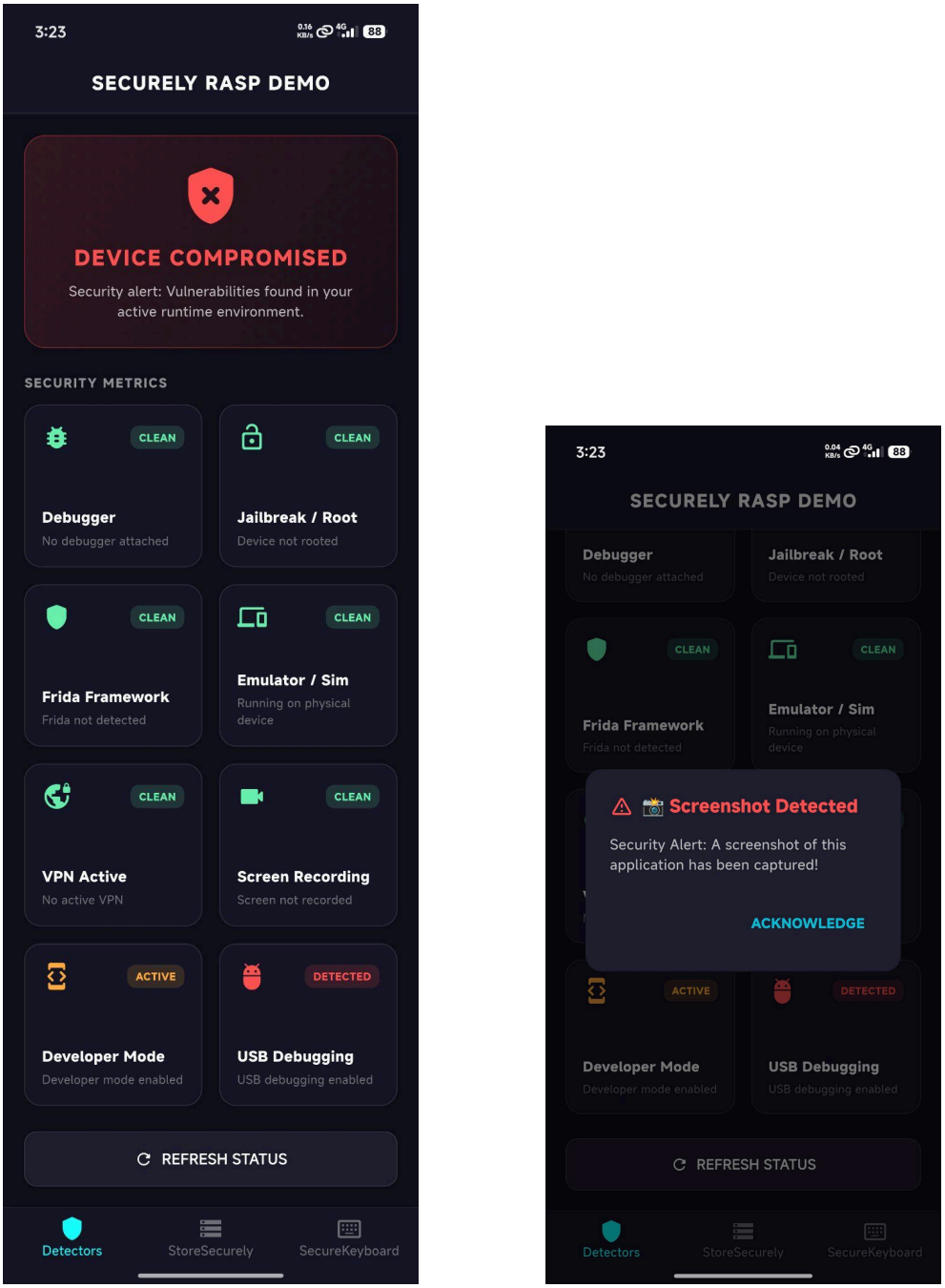

Figure-10: Threat Detection System, Example app of Securely.

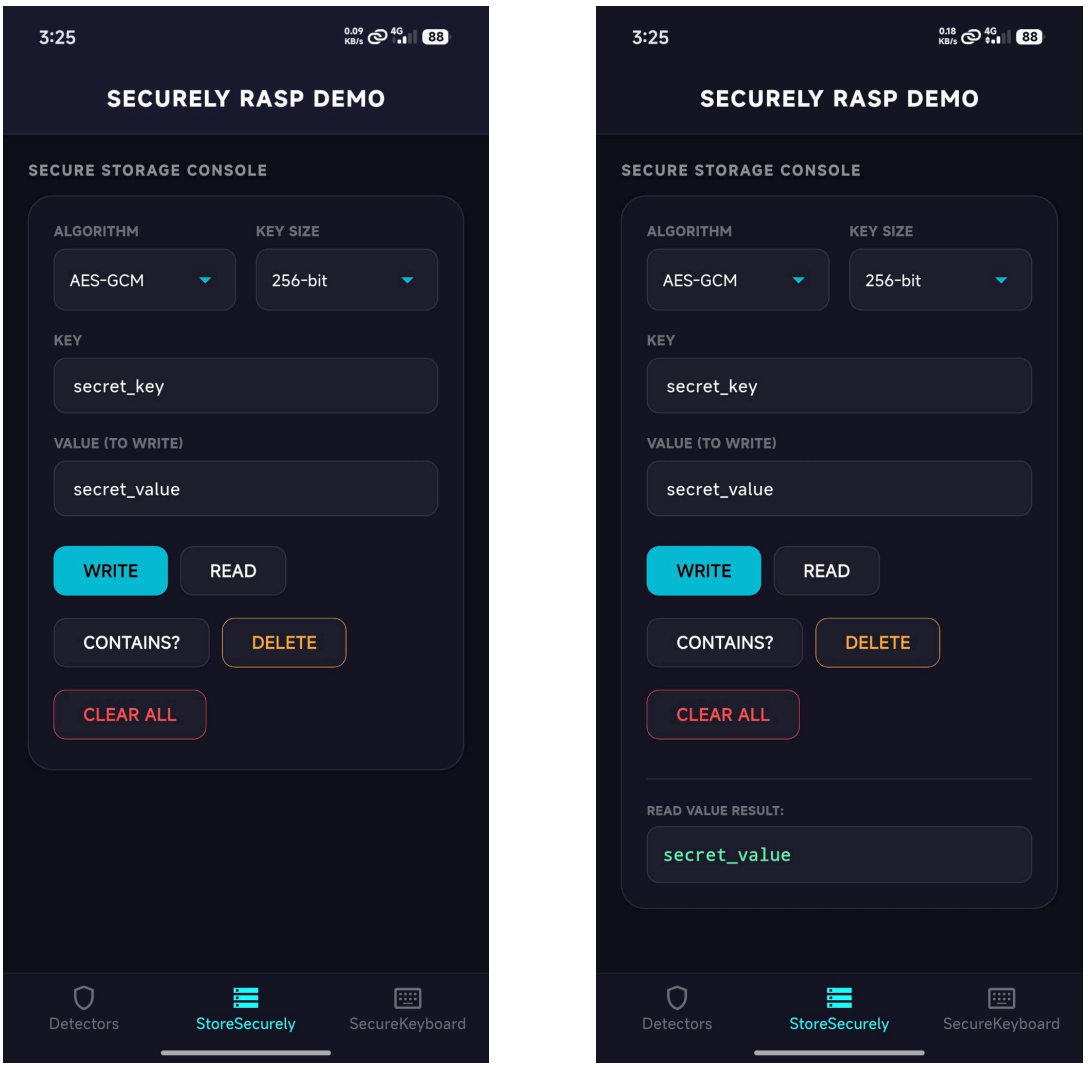

Figure-11: StoreSecurely, Example app of Securely.

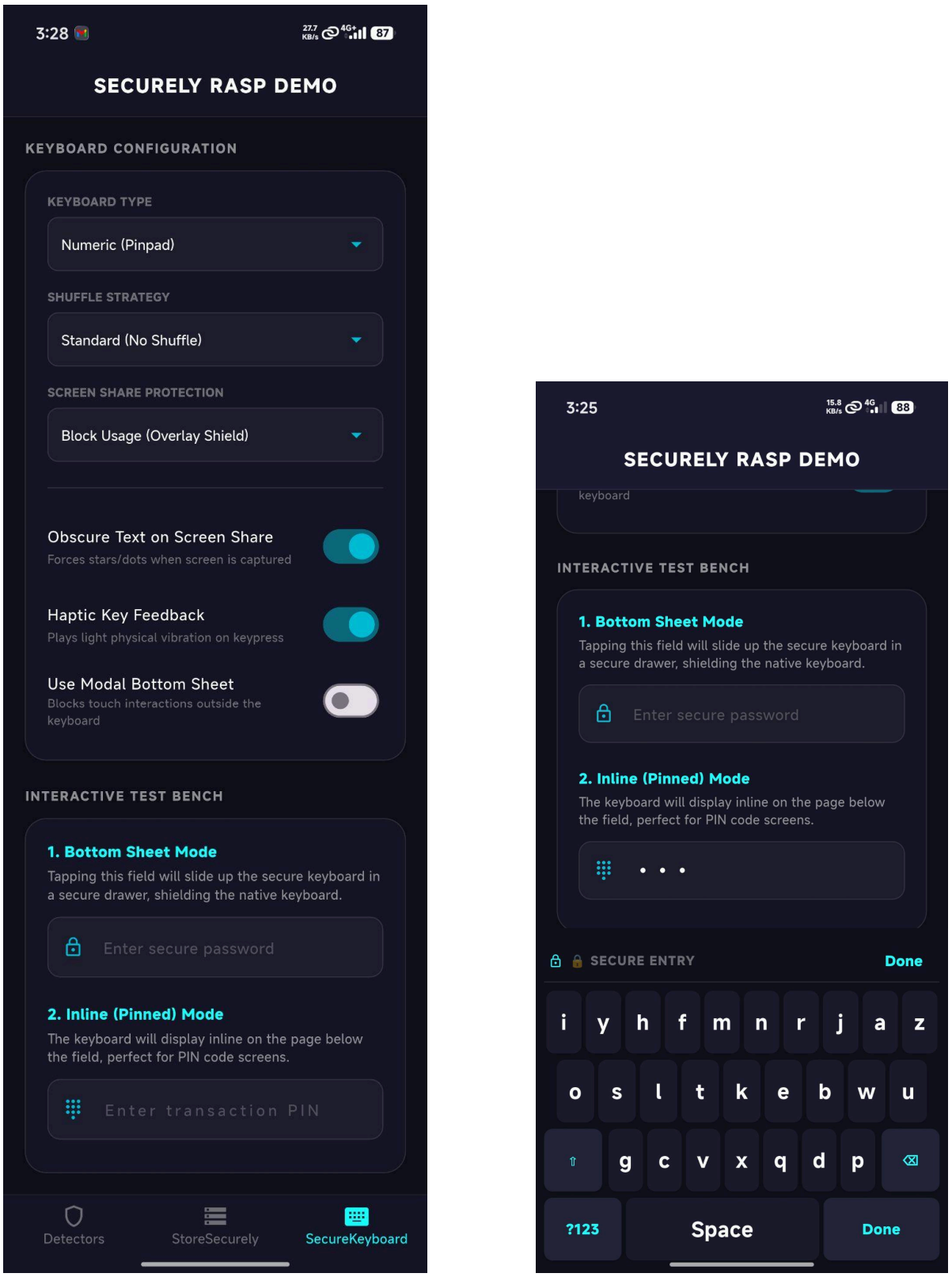

Figure-12: SecureKeyboard, Example app of Securely.

CHAPTER 6: PROJECT SUMMARY

1. Key Achievements

Unified Platform API: Successfully created a cross-platform RASP client covering five distinct target platforms (Android, iOS, macOS, Windows, Linux) alongside Web.

Zero-Clipboard Security: Built a custom Flutter UI text input suite that bypasses system soft keyboard overlays, neutralizing keylogger and clipboard vulnerabilities.

Custom Cryptographic Configurations: Constructed a secure storage library enabling GCM and CBC cryptographic selections using hardware chip security.

2. System Limitations

Platform Sandbox Limits: Telemetry checks on Web platforms cannot evaluate deep system components (e.g. jailbreak status, debug status) due to browser sandbox boundaries.

Evolving Bypass Tools: Attack vectors like customized root hide utilities or hardware-based HDMI screen grabbers require continuous detection updates.

3. Future Scope

Biometric Integration: Connect hardware storage keys with facial and fingerprint authentication.

Desktop Capturer Blocks: Add Windows-specific API blocks to restrict desktop-wide screenshots.

Advanced Telemetry: Incorporate heuristic system timing checks to detect emulator execution environments.

APPENDIX A: CODE DIRECTORY STRUCTURE

Below is the complete directory structure of the `securely` Flutter plugin package (extracted from the provided archive). This reflects the actual project layout as of version **1.1.0**.

```
securely/
├── .dart_tool/                # Dart build artifacts and package config
├── .gitignore
├── .metadata                  # Flutter project metadata
├── CHANGELOG.md
├── LICENSE                    # BSD-3-Clause
├── README.md
├── analysis_options.yaml
├── pubspec.lock
├── pubspec.yaml               # Package definition (v1.1.0)
│
├── android/                   # Android platform implementation
│   ├── build.gradle
│   └── src/
│       └── main/
│           ├── kotlin/me/alfuad/securely/
│           │   ├── SecurelyPlugin.kt        # Main RASP + storage logic
│           │   └── SecureStorageHelper.kt   # Android Keystore wrapper
│           └── res/...
│
├── ios/                       # iOS platform implementation
│   ├── securely.podspec
│   └── securely/
│       └── Sources/
│           └── securely/
│               ├── SecurelyPlugin.swift     # RASP checks (jailbreak, Frida, etc.)
│               └── PrivacyInfo.xcprivacy
│
├── macos/                     # macOS platform support
│   ├── securely.podspec
│   └── securely/
│       └── Sources/securely/...
│
├── linux/                     # Linux desktop implementation
│   ├── CMakeLists.txt
│   └── securely_plugin.cc     # C++ native plugin (RASP + storage)
│
├── windows/                   # Windows desktop implementation
│   └── securely_plugin.cpp
│
├── lib/                       # Dart/Flutter public API
│   ├── securely.dart          # Main API: Securely class + StoreSecurely
│   ├── securely_web.dart      # Web fallback implementation
│   └── src/
│       └── widgets/
│           ├── secure_keyboard.dart     # keyboard with shuffle & obscure modes
│           └── secure_text_field.dart   # SecureTextField widget
│
├── test/                      # Testing suite
│   ├── securely_test.dart             # Unit tests (RASP + storage)
│   └── secure_keyboard_test.dart      # Widget tests for keyboard
│
└── example/                   # Demo / test application
    ├── android/
    ├── ios/
    ├── macos/
    ├── linux/
    ├── windows/
    ├── lib/
    │   └── main.dart                  # Example app entry point
    ├── pubspec.yaml
    ├── integration_test/
    └── test/
```

Key Files Summary

Path	Description
lib/securely.dart	Core public API for RASP checks, secure storage, and event streams
lib/src/widgets/secure_keyboard.dart	Main secure input keyboard with shuffling and screen-share protection
lib/src/widgets/secure_text_field.dart	Widget that hijacks input to use the custom keyboard
android/src/main/kotlin/.../SecurelyPlugin.kt	Android RASP + Keystore implementation
ios/securely/Sources/securely/SecurelyPlugin.swift	iOS RASP (jailbreak, debugger, Frida, screen recording)
test/securely_test.dart & secure_keyboard_test.dart	Comprehensive unit and widget tests

This structure follows standard Flutter plugin conventions with full cross-platform support (Android, iOS, macOS, Linux, Windows, Web). All native code is properly isolated under platform-specific directories, while the Dart layer provides a unified, easy-to-use API.

APPENDIX B: PRINCIPAL CORE CODE IMPLEMENTATIONS

This appendix presents key excerpts from the core implementation files of the securely Flutter plugin. Full source code is available in the project repository.

1. Main Dart API (lib/securely.dart)

```
  class Securely {
    static const MethodChannel _channel = MethodChannel('securely');

    static final StreamController<void> _screenshotController =
StreamController<void>.broadcast();
    static final StreamController<bool> _screenRecordingController =
StreamController<bool>.broadcast();
    static bool _initialized = false;

    static void _initialize() {
      if (_initialized) return;
      _initialized = true;
      _channel.setMethodCallHandler((MethodCall call) async {
        switch (call.method) {
          case 'onScreenshotTaken':
            _screenshotController.add(null);
            break;
          case 'onScreenRecordingChanged':
            _screenshotController.add(null); // Triggers screenshot stream callback
            _screenRecordingController.add(call.arguments as bool);
            break;
        }
      });
    }

    static Future<bool> isDebuggerDetected() async => await
_channel.invokeMethod('isDebuggerDetected');
    static Future<bool> isRootDetected() async => await
_channel.invokeMethod('isRootDetected');
    static Future<bool> isEmulatorDetected() async => await
_channel.invokeMethod('isEmulatorDetected');
    static Future<bool> isFridaDetected() async => await
_channel.invokeMethod('isFridaDetected');
    static Future<bool> isVpnDetected() async => await
_channel.invokeMethod('isVpnDetected');
  }
```

2. Secure Keyboard Layout Scrambler (lib/src/widgets/secure_keyboard.dart)

```
void _shuffleKeys() {
  final rand = math.Random();
  // Shuffle numeric key mapping layout using modern Fisher-Yates shuffle
  for (int i = _numbers.length - 1; i > 0; i--) {
    int j = rand.nextInt(i + 1);
    final temp = _numbers[i];
    _numbers[i] = _numbers[j];
    _numbers[j] = temp;
  }
  // Shuffle alphanumeric key mapping layout
  for (int i = _letters.length - 1; i > 0; i--) {
    int j = rand.nextInt(i + 1);
    final temp = _letters[i];
    _letters[i] = _letters[j];
    _letters[j] = temp;
  }
}
```

3. Keyboard Hijack Prevention & Obfuscation (lib/src/widgets/secure_text_field.dart)

```
  @override
  Widget build(BuildContext context) {
    // Obscure input characters if standard configuration is set OR screen sharing is detected
    final shouldObscure = widget.obscureText || (_isScreenRecording &&
widget.obscureOnScreenShare);
    final isBlocked = _isScreenRecording && widget.keyboardObscureMode ==
SecureKeyboardObscureMode.blockKeyboard;
    final isFieldEnabled = widget.enabled && !isBlocked;

    return TextField(
      controller: widget.controller,
      focusNode: _focusNode,
      enabled: isFieldEnabled,
      obscureText: shouldObscure,
      obscuringCharacter: widget.obscuringCharacter,

      // Core parameters to override the default OS soft keyboard rendering:
      readOnly: true,       // Suppresses default soft keyboard popups
      showCursor: true,     // Retains blinking text cursor for normal user experience
      enableInteractiveSelection: false, // Disables text selection overlay options
(Copy/Paste/Cut)

      onTap: () {
        if (widget.onTap != null) widget.onTap!();
        if (widget.showKeyboardBottomSheet && !_isBottomSheetOpen && isFieldEnabled)
{
          _showKeyboardBottomSheet();
        }
      },
    );
  }
```

4. Hardware-Backed Android Keystore Helper (SecureStorageHelper.kt)

```
  @Synchronized
  private fun getOrCreateSecretKey(algorithm: String, keySize: String): SecretKey {
      val sizeBits = if (keySize == "bits128") 128 else 256
      val keyAlias = "securely_key_${algorithm}_$keySize"

      if (keyStore.containsAlias(keyAlias)) {
          val entry = keyStore.getEntry(keyAlias, null) as? KeyStore.SecretKeyEntry
          if (entry != null) return entry.secretKey
      }
```

```
        val keyGenerator =
KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES,
ANDROID_KEYSTORE)
        val blockMode = if (algorithm == "aesCbc") KeyProperties.BLOCK_MODE_CBC else
KeyProperties.BLOCK_MODE_GCM
        val padding = if (algorithm == "aesCbc")
KeyProperties.ENCRYPTION_PADDING_PKCS7 else
KeyProperties.ENCRYPTION_PADDING_NONE

        val spec = KeyGenParameterSpec.Builder(keyAlias,
KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
            .setKeySize(sizeBits)
            .setBlockModes(blockMode)
            .setEncryptionPaddings(padding)
            .setRandomizedEncryptionRequired(true)
            .build()
        keyGenerator.init(spec)
        return keyGenerator.generateKey()
    }
```

5. Linux-Style Frida Memory Mapping Scan (SecurelyPlugin.kt)

```
private fun isFridaDetected(): Boolean {
    val suspiciousLibs = listOf("frida", "gum-js-loop", "gadget")
    return try {
        val maps = File("/proc/self/maps")
        if (!maps.exists()) return false
        // Search current process memory maps for dynamic hooks
        maps.readLines().any { line ->
            suspiciousLibs.any { lib -> line.contains(lib, ignoreCase = true) }
        }
    } catch (e: Exception) {
        false
    }
}
```

6. iOS Native System Debugger Attachment Check (SecurelyPlugin.swift)

```
private func isDebuggerDetected() -> Bool {
    var info = kinfo_proc()
    var size = MemoryLayout<kinfo_proc>.stride
    var name = [CTL_KERN, KERN_PROC, KERN_PROC_PID, getpid()]

    // Query active process flags from kernel using sysctl
    let sysctlResult = sysctl(&name, 4, &info, &size, nil, 0)
    if sysctlResult != 0 {
        return false
    }

    // Check if the P_TRACED flag is set on the process
    return (info.kp_proc.p_flag & P_TRACED) != 0
}
```